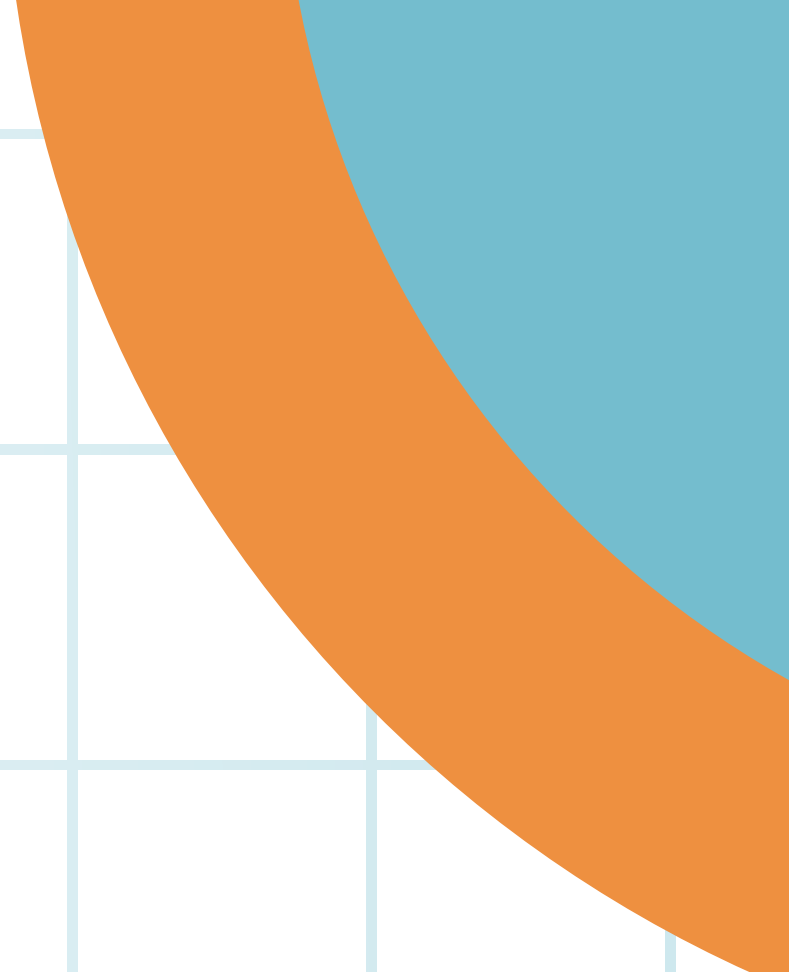
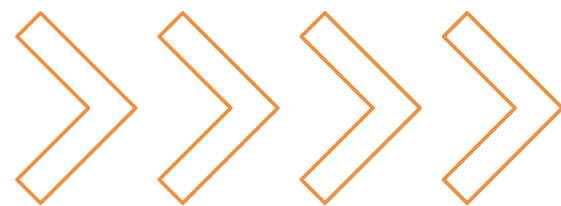
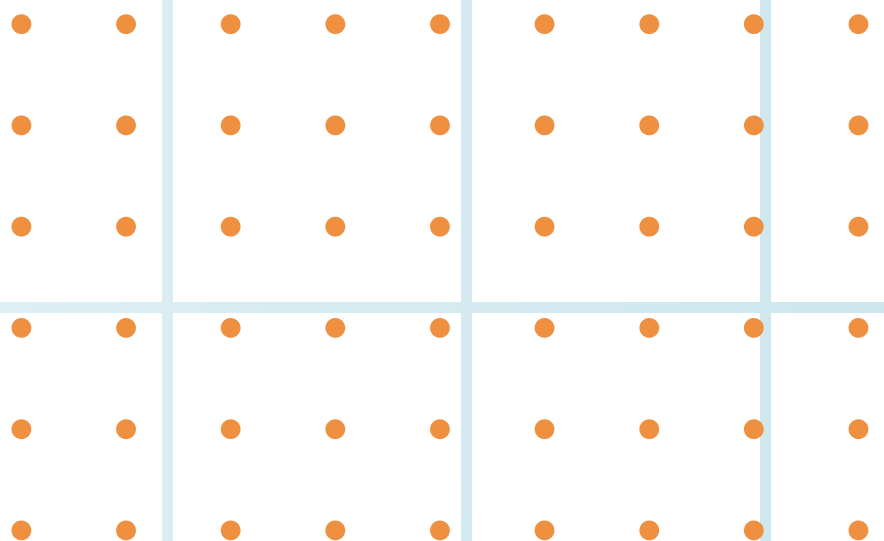




IMPLEMENTAÇÃO DE DATA LAKEHOUSE COM DELTA LAKE



Prof. Adnan Jebailey



Aula 05

Qualidade, Otimização e Manutenção



Qualidade

Expectations

Not Null e Check

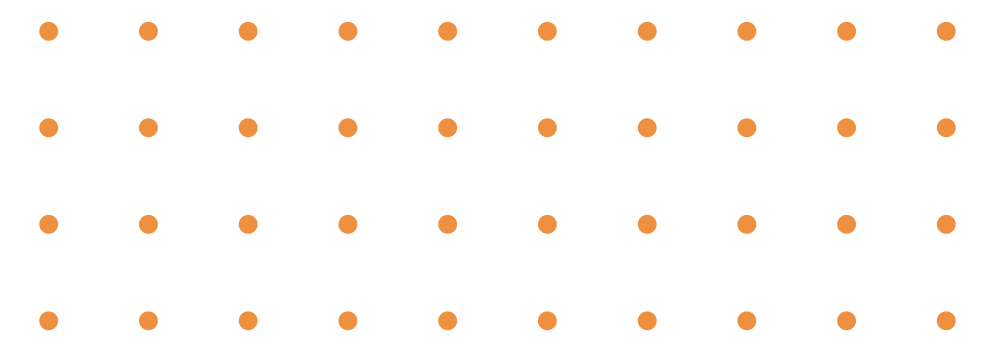


Not Null: Garante que os dados de uma coluna específica em uma tabela não sejam nulo.

Check: Uma expressão booleana que deve ser avaliada como Verdadeira para cada linha antes de ser inserida. As restrições de verificação permitem que os engenheiros de dados apliquem uma lógica de validação complexa que uma coluna específica deve satisfazer.

(GIRTEN, 2024)

Expectations



As expectations são regras de qualidade de dados definidas juntamente com a definição de um conjunto de dados em um pipeline DLT. A regra de qualidade de dados é uma expressão booleana aplicada a cada registro que passa por uma determinada definição de conjunto de dados. A expressão deve ser avaliada como Verdadeira para que o registro seja marcado como aprovado; caso contrário, resultará em um registro reprovado, indicando que o registro não passou na validação de qualidade de dados.

Além disso, o pipeline DLT registrará métricas de qualidade de dados para cada linha processada. Por exemplo, o DLT registrará o número de registros que passaram na validação de qualidade de dados, bem como o número de registros que não passaram.

(GIRTEN, 2024)

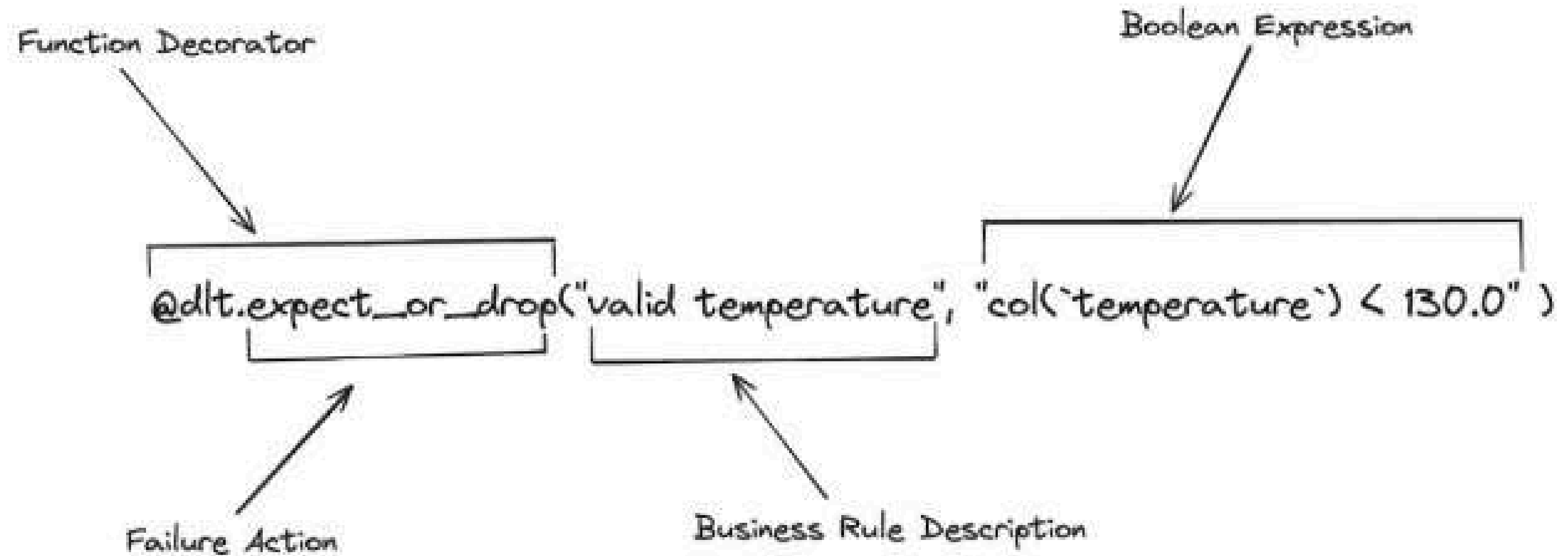


Figura 3.2 – Os principais componentes de uma expectativa de DLT

(GIRTEN, 2024)

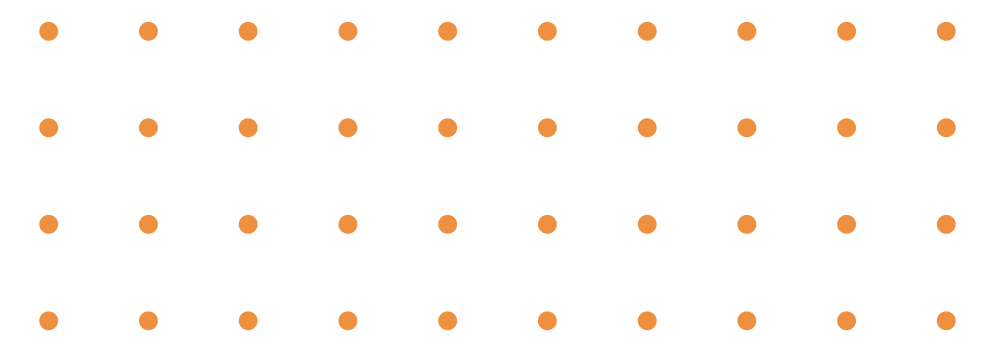
Expectations



Por padrão, o sistema DLT simplesmente registrará que a linha falhou na expressão booleana para uma linha específica nos logs do sistema e registrará as métricas de qualidade de dados no sistema.

(GIRTEN, 2024)

Expectations



Existem três tipos de ações que a tecnologia DLT pode ser acionada quando um registro específico viola as restrições de dados definidas em um conjunto de dados DLT:

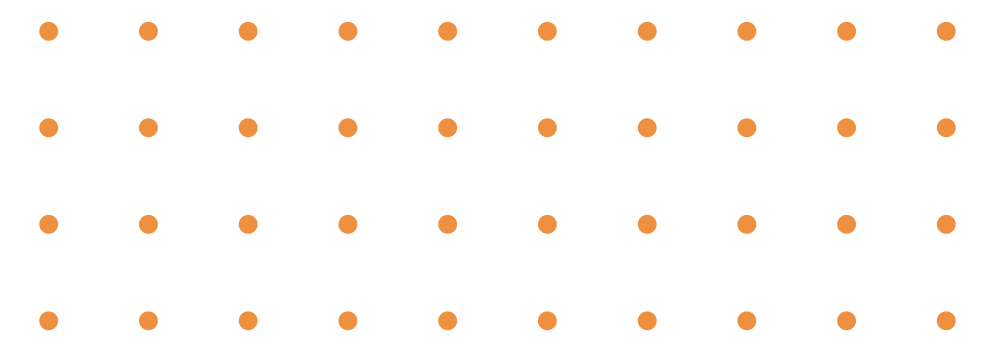
Warn (Aviso): Quando o DLT encontra uma violação de expressão, o registro será gravado como uma métrica e continuará sendo gravado no conjunto de dados de destino subsequente

Drop (Descartar): Quando o DLT encontra uma violação de expressão, o registro será gravado como uma métrica e será impedido de entrar no conjunto de dados de destino subsequente

Fail (Falha): Quando a DLT encontra uma violação de expressão, a atualização do pipeline falhará completamente até que um membro da equipe de engenharia de dados possa investigar e corrigir a violação de dados ou possível corrupção de dados

(GIRTEN, 2024)

Múltiplos Expectations



Pode haver momentos em que um o autor do conjunto de dados pode querer aplicar mais de uma regra de negócio ou restrição de qualidade de dados em cada linha do conjunto. Nesse caso, o DLT fornece um conjunto especial de decoradores de função para especificar várias restrições de qualidade de dados.

O decorador de função `@dlt.expect_all()` pode ser usado para combinar mais de uma restrição de qualidade de dados para um conjunto de dados específico. Da mesma forma, `expect_all_or_drop()` pode ser especificado quando os dados recebidos devem ser descartados de entrar em uma tabela de destino, a menos que todos os critérios no conjunto de restrições de qualidade de dados sejam atendidos. Por fim, `expect_all_or_fail()` interromperá a execução de um pipeline de dados se algum dos critérios em um conjunto de restrições de qualidade de dados não for atendido pelos dados recebidos .

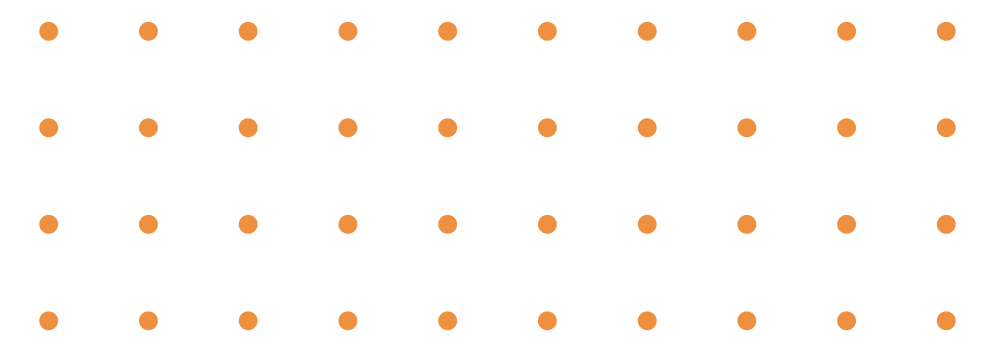
(GIRTEN, 2024)



Otimização e Manutenção

**Auto compactação, Propriedades de Manutenção e
Definições de Esquema**

Auto Compactação



[...] o Delta Lake oferece um comando útil chamado OPTIMIZE para realizar essas tarefas de manutenção. Por padrão, o comando OPTIMIZE do Delta Lake tentará consolidar arquivos menores em arquivos maiores de 1 gigabyte.

(GIRTEN, 2024)

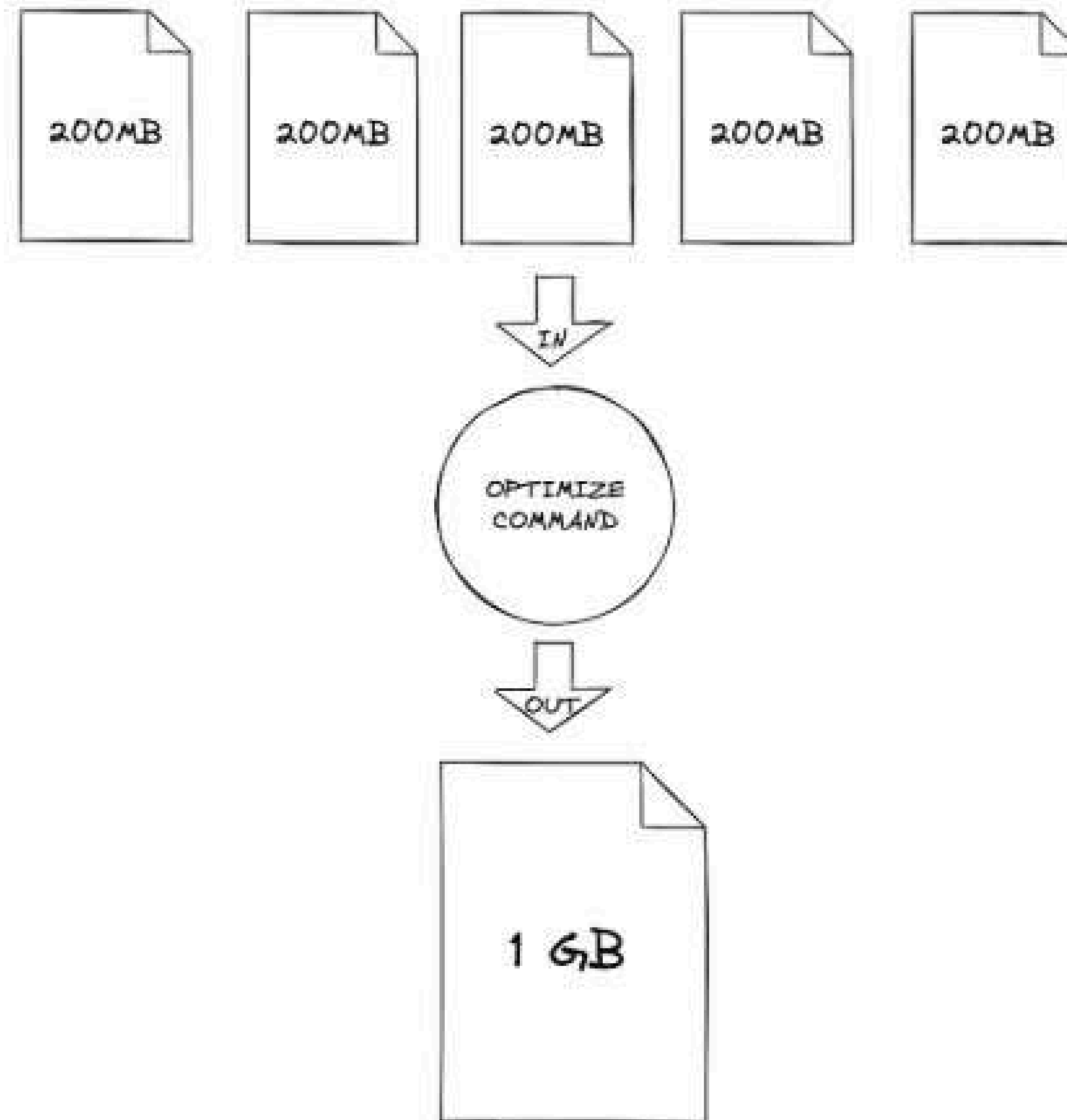
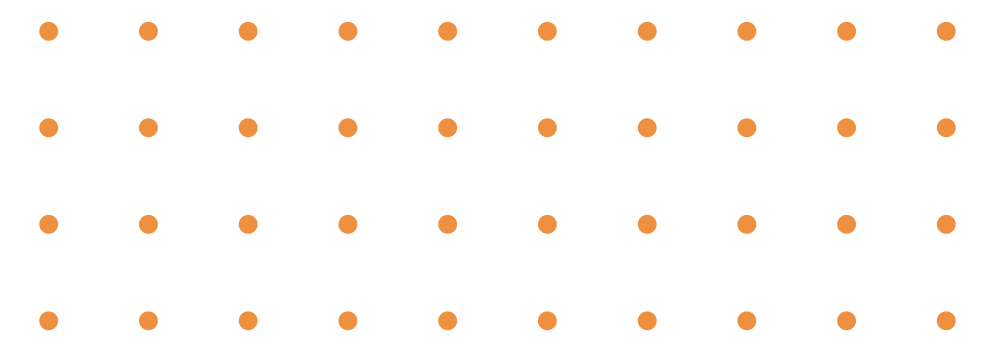


Figura 4.2 – O DLT executará automaticamente o comando OPTIMIZE nas tabelas

Propriedades de Manutenção



O Delta Lake oferece diversas funções utilitárias para auxiliar na manutenção geral (limpeza e otimização), reparo, restauração e até mesmo substituição de nossas tabelas críticas, sendo todas essas funcionalidades valiosas para qualquer engenheiro de dados.

A grande vantagem de usar propriedades de tabela é que elas afetam apenas os metadados das tabelas e, na maioria dos casos, não exigem alterações na estrutura física da tabela. Além disso, a possibilidade de ativar ou desativar essas propriedades nos permite modificar o comportamento do Delta Lake sem precisar alterar o código existente do pipeline e, na maioria dos casos, sem precisar reiniciar ou reimplantar nossos aplicativos de streaming (os aplicativos em lote simplesmente lerão as propriedades revisadas na próxima execução).

(LEE, 2024)

Tabela de Propriedades

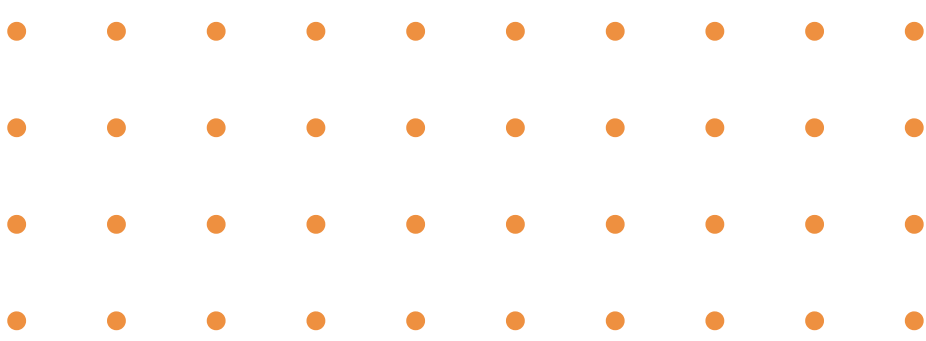
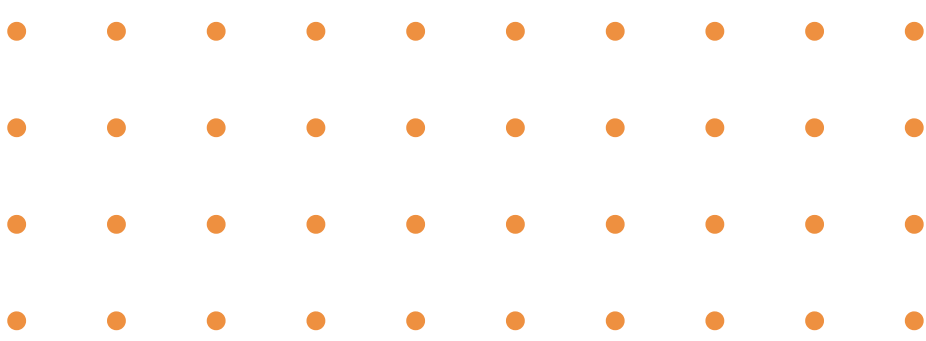


Table 5-1. Delta Lake table properties reference

Property	Data type	Use with	Default
<code>delta.logRetentionDuration</code>	CalendarInterval	Cleaning	interval 30 days
<code>delta.deletedFileRetentionDuration</code>	CalendarInterval	Cleaning	interval 1 week
<code>delta.setTransactionRetentionDuration</code>	CalendarInterval	Cleaning, Repairing	(none)
<code>delta.targetFileSize</code> ^a	String	Tuning	(none)
<code>delta.tuneFileSizesForRewrites</code> ^a	Boolean	Tuning	(none)

Tabela de Propriedades



delta.autoOptimize.optimizeWrite	Boolean	Tuning	(none)
delta.autoOptimize.autoCompact	Boolean	Tuning	(none)
delta.dataSkippingNumIndexedCols	Int	Tuning	32
delta.checkpoint.writeStatsAsStruct	Boolean	Tuning	(none)
delta.checkpoint.writeStatsAsJson	Boolean	Tuning	true
delta.randomizeFilePrefixes	Boolean	Tuning	false

Propriedades



delta.logRetentionDuration — Define por quanto tempo o Delta Lake mantém os logs de transações (`_delta_log`) antes de serem removidos pelo VACUUM/limpeza.

delta.deletedFileRetentionDuration — Define por quanto tempo arquivos de dados que foram deletados ou substituídos ficam disponíveis antes de poderem ser removidos pelo VACUUM.

delta.setTransactionRetentionDuration — Define por quanto tempo informações de transações de escrita ficam registradas para evitar que operações idempotentes sejam reaplicadas.

delta.targetFileSize — Define o tamanho-alvo dos arquivos Parquet gerados pelo Delta durante operações de escrita/otimização.

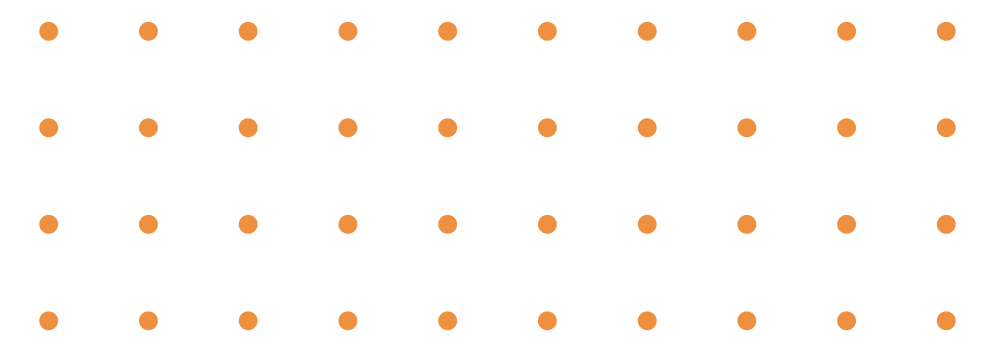
delta.tuneFileSizesForRewrites — Permite que o Delta ajuste dinamicamente o tamanho dos arquivos durante operações de reescrita, buscando otimizar o desempenho.

delta.autoOptimize.optimizeWrite — Faz o Delta otimizar a escrita para reduzir a criação de arquivos pequenos (small files).

Propriedades

- **delta.autoOptimize.autoCompact** — Faz o Delta executar compactação automática de arquivos pequenos após as escritas.
- **delta.dataSkippingNumIndexedCols** — Define quantas colunas terão estatísticas indexadas para permitir data skipping e evitar a leitura de arquivos que não podem conter os dados procurados.
- **delta.checkpoint.writeStatsAsStruct** — Faz as estatísticas dos arquivos serem armazenadas no checkpoint em formato estruturado (struct).
- **delta.checkpoint.writeStatsAsJson** — Faz as estatísticas dos arquivos serem armazenadas no checkpoint também em formato JSON.
- **delta.randomizeFilePrefixes** — Randomiza os prefixos dos nomes dos arquivos, ajudando a distribuir as operações de I/O quando o armazenamento apresenta problemas com muitos arquivos usando o mesmo prefixo.

Definições do Schema

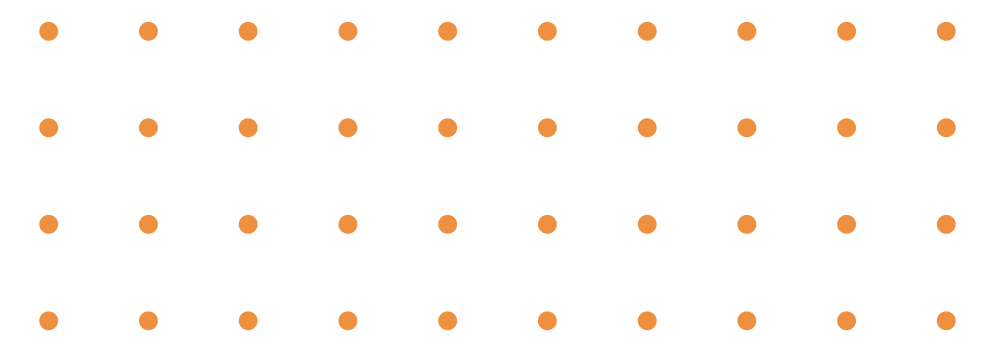


Schema enforcement: É o processo de controle que verifica um esquema existente antes de permitir que uma transação de escrita ocorra e resulta no lançamento de uma exceção em caso de incompatibilidade.

Schema evolution: Esse é o processo de modificar intencionalmente um esquema existente de forma a permitir a retrocompatibilidade. Tradicionalmente, isso é realizado usando `ALTER TABLE {t} ADD COLUMN(S)`, que também é suportado no Delta Lake, juntamente com a capacidade de habilitar a mergeSchema opção na gravação.

(LEE, 2024)

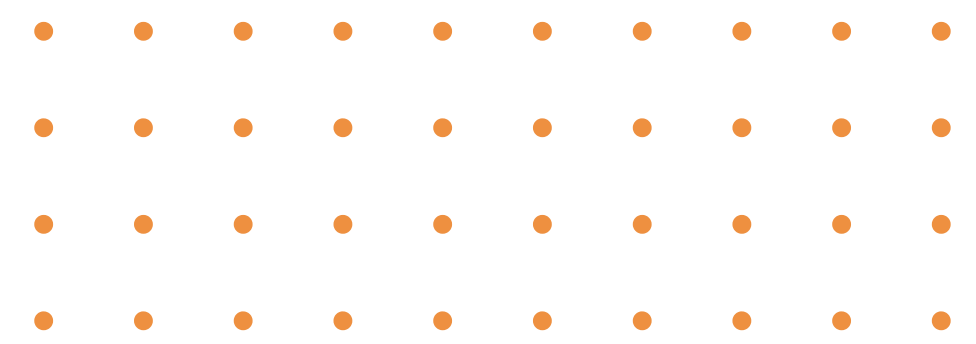
Evolução do Schema



[...] `option("mergeSchema", "true")` para modificar o comportamento do gravador Delta Lake. Embora essa opção simplifique a evolução do esquema da tabela Delta Lake, ela tem o custo de não termos total conhecimento das alterações no esquema da tabela. Em casos em que colunas desconhecidas são introduzidas de uma fonte upstream, é importante saber quais colunas devem ser importadas e quais podem ser ignoradas com segurança.

(LEE, 2024)

Substituição, Restauração e Vácuo

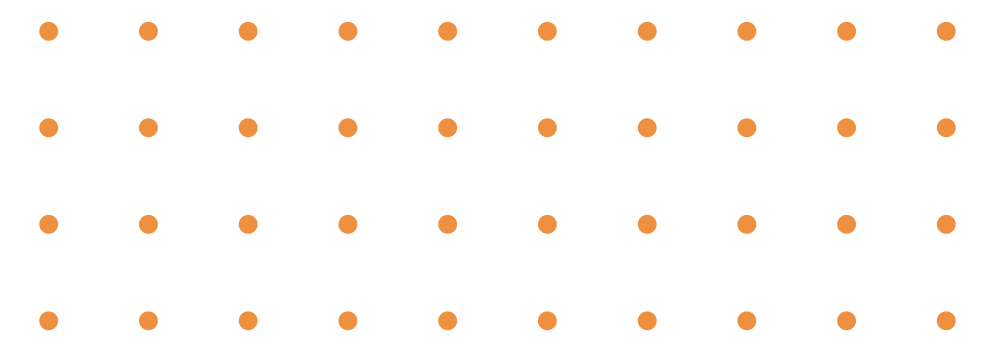


Substituição

[...] o padrão de substituição com sobrescrita, que pode tanto substituir dados ausentes quanto sobrescrever dados existentes condicionalmente em uma tabela. Essa opção permite corrigir tabelas que possam ter sido corrompidas ou resolver problemas em que dados estavam ausentes e agora estão disponíveis. O comando ``replaceWhere`with insert overwrite`` não se limita apenas a colunas de partição e pode ser usado para substituir dados condicionalmente em suas tabelas

(LEE, 2024)

Substituição, Restauração e Vácuo

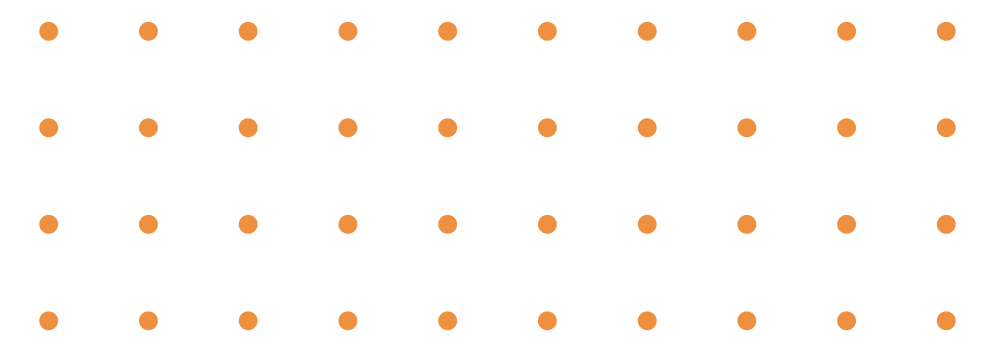


Restauração

No caso de uma transação ter ocorrido — por exemplo, uma exclusão incorreta da sua tabela (porque imprevistos acontecem) — em vez de recarregar os dados (caso tenhamos uma cópia dos dados), podemos retroceder e restaurar a tabela para uma versão anterior. Essa é uma capacidade importante, especialmente considerando que podem surgir problemas quando a única cópia dos seus dados era justamente a que acabou de ser excluída. Quando não há mais para onde recorrer para recuperar os dados, você pode viajar no tempo para uma versão anterior da sua tabela.

(LEE, 2024)

Substituição, Restauração e Vácuo

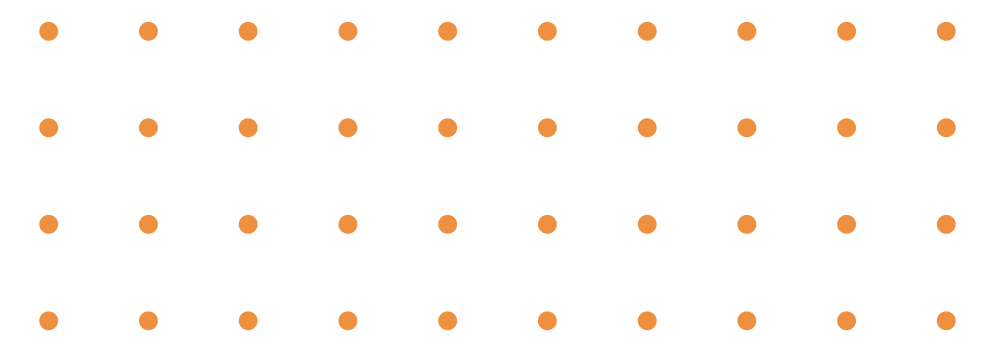


Vácuo

O vacuum comandará limpar arquivos excluídos ou versões da tabela que não são mais atuais, o que pode acontecer quando você usa o método de sobrescrita em uma tabela. Se você sobrescrever a tabela, tudo o que você está realmente fazendo é criar novos ponteiros para novos arquivos que são referenciados pelos metadados da tabela. Portanto, se você sobrescrever uma tabela com frequência, o tamanho da tabela em disco crescerá exponencialmente. Com isso em mente, é melhor utilizar vacuumo recurso de viagem no tempo de curta duração (até 30 dias é típico) e empregar uma estratégia diferente para armazenar backups estratégicos da tabela.

(LEE, 2024)

Substituição, Restauração e Vácuo



Vácuo

Executar o comando VACUUMem nossa tabela resultará na remoção de todos os arquivos que não são mais referenciados pelo snapshot da tabela, incluindo arquivos excluídos de versões anteriores. Embora o VACUUM seja um processo necessário para reduzir o custo de manutenção de versões antigas de uma tabela, ele tem um efeito colateral: os usuários de dados subsequentes podem ficar sem acesso a uma versão antiga da tabela caso precisem lê-la.

(LEE, 2024)

Referências

REFERÊNCIAS BIBLIOGRÁFICAS

GIRTEN, Will. Building modern data applications using Databricks Lakehouse: develop, optimize, and monitor data pipelines on Databricks. Birmingham: Packt Publishing, 2024. 246 p. ISBN 978-1-80107-323-3.

LEE, Denny; WENTLING, Tristen; HAINES, Scott; BABU, Prashanth. Delta Lake: the definitive guide: modern data lakehouse architectures with data lakes. Sebastopol: O'Reilly Media, 2024. 382 p. ISBN 978-1-098-15194-2.